Experiment 3: Implementing Artificial Neural Networks (ANNs) Without and With Backpropagation

Question

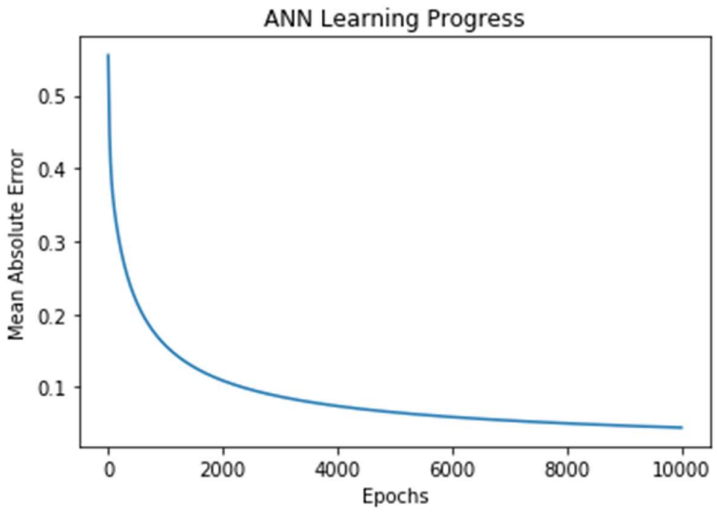
To implement an artificial neural network (ANN) model both with and without backpropagation to observe its learning process and efficiency.

Theory

Artificial Neural Networks (ANNs) are computing models inspired by the human brain. They consist of input, hidden, and output layers and use weights and activation functions to process data. Backpropagation is a supervised learning technique that updates weights based on the error gradient to improve learning efficiency.

Implementation:

```
import numpy as np
import matplotlib.pyplot as plt

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of Sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Training Data (AND Function)
X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([[0], [0], [0], [1]])

# Initialize weights and bias
weights = np.random.rand(2,1)
bias = np.random.rand(1)
learning_rate = 0.1
epochs = 10000
errors = []

for epoch in range(epochs):
    weighted_sum = np.dot(X, weights) + bias
    output = sigmoid(weighted_sum)
    error = y - output
```

```
    errors.append(np.mean(np.abs(error)))
    adjustments = error * sigmoid_derivative(output)
    weights += np.dot(X.T, adjustments) * learning_rate
    bias += np.sum(adjustments) * learning_rate

# Plot error reduction over epochs
plt.plot(errors)
plt.xlabel('Epochs')
plt.ylabel('Mean Absolute Error')
plt.title('ANN Learning Progress')
plt.show()
```

Output